

Authored by: Arpit Raj, LNMIIT Jaipur

Date: August 27, 2026

Copyright: © 2026 Arpit Raj

LC 3429 - Paint House IV

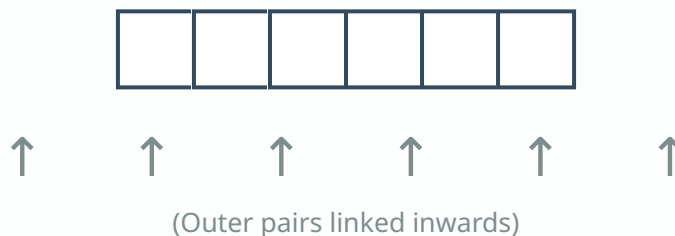
Problem Intuition & Rules

- **n Houses**
- Costs vector is basically $[n \times 3] \rightarrow 3 \text{ available colors}$

Houses look to aadya if :-

- No 2 adjacent houses have the same color.
- Equidistant houses do not have the same color.

i.e.



these pairs not same color

Need **minimum cost** to make pattern ethereal (Aadya).

Dynamic Programming Approach

- $\rightarrow$ It's given **n is even**
- So we can go from **outer pairs to inner pairs**.
- Let's say $dp[left\ col][right\ col]$ for the outermost pair.

- And then as we move to inner pairs, we maintain `ndp[newleft][newright]`.
- And before going to the next inner pair:
`dp = ndp` (parent level reduces).

C++ Implementation

```
class Solution {
public:
    long long minCost(int n, vector<vector<int>>& cost) {

        vector<vector<long long>> dp(3, vector<long long>(3,
LLONG_MAX));
        // basically dp[leftccol][rightcol]
        // n even he to outer to inner jaayenge
        // sbse pehle 0 aur n-1 pair handle

        for(int lc = 0; lc < 3; lc++) {
            for(int rc = 0; rc < 3; rc++) {
                if(lc == rc) continue;
                dp[lc][rc] = cost[0][lc] + cost[n-1][rc];
            }
        }

        for(int l = 1; l < n/2; l++) {
            int r = n - l - 1;

            vector<vector<long long>> ndp(3, vector<long long>(3,
LLONG_MAX));
            // dp abhi bhi chahiye kyunki jo cost hoga wo dp ke cost me
            // hi to add hoga
            for(int prevl = 0; prevl < 3; prevl++) {
                for(int prevr = 0; prevr < 3; prevr++) {

                    for(int newl = 0; newl < 3; newl++) {
                        if(prevl == newl) continue;

                        for(int newr = 0; newr < 3; newr++) {
                            if(newr == prevr) continue;
                            if(newr == newl) continue;

                            if(dp[prevl][prevr] == LLONG_MAX) continue;
                            ndp[newl][newr] = min(ndp[newl][newr],
dp[prevl][prevr] + cost[l][newl] + cost[r][newr]);
                        }
                    }
                }
            }
        }
    }
};
```

```

        }
    }
}
dp = ndp;
}

long long ans = LLONG_MAX;
for(int lc = 0; lc < 3; lc++) {
    for(int rc = 0; rc < 3; rc++) {
        ans = min(ans, dp[lc][rc]);
    }
}

if(ans != LLONG_MAX)
    return ans;
else
    return -1;
}
};

```